

运输计划(transport.cpp)

【问题描述】

公元 2044 年，人类进入了宇宙纪元。

L 国有 n 个星球，还有 $n-1$ 条双向航道，每条航道建立在两个星球之间，这 $n-1$ 条航道连通了 L 国的所有星球。

小 P 掌管一家物流公司，该公司有很多个运输计划，每个运输计划形如：有一艘物流飞船需要从 u_i 号星球沿最快的宇航路径飞行到 v_i 号星球去。显然，飞船驶过一条航道是需要时间的，对于航道 j ，任意飞船驶过它所花费的时间为 t_j ，并且任意两艘飞船之间不会产生任何干扰。

为了鼓励科技创新，L 国国王同意小 P 的物流公司参与 L 国的航道建设，即允许小 P 把某一条航道改造成虫洞，飞船驶过虫洞不消耗时间。

在虫洞的建设完成前小 P 的物流公司就预接了 m 个运输计划。在虫洞建设完成后，这 m 个运输计划会同时开始，所有飞船一起出发。当这 m 个运输计划都完成时，小 P 的物流公司的阶段性工作就完成了。

如果小 P 可以自由选择将哪一条航道改造成虫洞，试求出小 P 的物流公司完成阶段性工作所需要的最短时间是多少？

【输入格式】

第一行包括两个正整数 n, m ，表示 L 国中星球的数量及小 P 公司预接的运输计划的数量，星球从 1 到 n 编号。

接下来 $n-1$ 行描述航道的建设情况，其中第 i 行包含三个整数 a_i, b_i 和 t_i ，表示第 i 条双向航道修建在 a_i 与 b_i 两个星球之间，任意飞船驶过它所花费的时间为 t_i 。

数据保证

接下来 m 行描述运输计划的情况，其中第 j 行包含两个正整数 u_j 和 v_j ，表示第 j 个运输计划是从 u_j 号星球飞往 v_j 号星球。

【输出格式】

一个整数，表示小 P 的物流公司完成阶段性工作所需要的最短时间。

【输入样例】

6 3
1 2 3
1 6 4
3 1 7
4 3 6
3 5 5
3 6
2 5
4 5

【输出样例】

11

【数据范围】

所有测试数据的范围和特点如下表所示

【数据规模与约定】

所有测试数据的范围和特点如下表所示

测试点编号	n=	m=	约定
1	100	1	第 i 条航道连接 i 号星球与 i+1 号星球
2		100	
3		100	
4	2000	1	第 i 条航道连接 i 号星球与 i+1 号星球
5	1000	1000	
6	2000	2000	
7	3000	3000	第 i 条航道连接 i 号星球与 i+1 号星球
8	1000	1000	
9	2000	2000	
10	3000	3000	第 i 条航道连接 i 号星球与 i+1 号星球
11	80000	1	
12	100000	1	
13	70000	70000	第 i 条航道连接 i 号星球与 i+1 号星球
14	80000	80000	
15	90000	90000	
16	100000	100000	第 i 条航道连接 i 号星球与 i+1 号星球
17	80000	80000	
18	90000	90000	
19	100000	100000	第 i 条航道连接 i 号星球与 i+1 号星球
20	300000	300000	
所有数据			$1 \leq a_i, b_i, u_i, v_i \leq n, 0 \leq t_i \leq 1000$

请注意常数因子带来的程序效率上的影响。

请注意常数因子带来的程序效率上的影响。

对于 100% 的数据，保证： $1 \leq a_i, b_i \leq n, 0 \leq t_i \leq 1000, 1 \leq u_i, v_i \leq n$ 。

开车旅行(drive.cpp)

【问题描述】

小 A 和小 B 决定利用假期外出旅行，他们将想去的城市从 1 到 n 编号，且编号较小的城市在编号较大的城市的西边，已知各个城市的海拔高度互不相同，记城市 i 的海拔高度为 h_i ，城市 i 和城市 j 之间的距离 $d_{i,j}$ 恰好是这两个城市海拔高度之差的绝对值，即 $d_{i,j} = |h_i - h_j|$ 。

旅行过程中，小 A 和小 B 轮流开车，第一天小 A 开车，之后每天轮换一次。他们计划选择一个城市 s 作为起点，一直向东行驶，并且最多行驶 x 公里就结束旅行。

小 A 和小 B 的驾驶风格不同，小 B 总是沿着前进方向选择一个最近的城市作为目的地，而小 A 总是沿着前进方向选择第二近的城市作为目的地（注意：本题中如果当前城市到两个城市的距离相同，则认为离海拔低的那个城市更近）。如果其中任何一人无法按照自己的原则选择目的城市，或者到达目的地会使行驶的总距离超出 x 公里，他们就会结束旅行。

在启程之前，小 A 想知道两个问题：

1、对于一个给定的 $x=x_0$ ，从哪一个城市出发，小 A 开车行驶的路程总数与小 B 行驶的路程总数的比值最小（如果小 B 的行驶路程为 0，此时的比值可视为无穷大，且两个无穷大视为相等）。如果从多个城市出发，小 A 开车行驶的路程总数与小 B 行驶的路程总数的比值都最小，则输出海拔最高的那个城市。

2、对任意给定的 $x=x_i$ 和出发城市 s_i ，小 A 开车行驶的路程总数以及小 B 行驶的路程总数。

【输入格式】

第一行包含一个整数 n ，表示城市的数目。

第二行有 n 个整数，每两个整数之间用一个空格隔开，依次表示城市 1 到城市 n 的海拔高度，即 $h_1, h_2, \dots, h_n$ ，且每个 h_i 都是互不相同的。

第三行包含一个整数 x_0 。

第四行为一个整数 m ，表示给定 m 组 s_i 和 x_i 。

接下来的 m 行，每行包含 2 个整数 s_i 和 x_i ，表示从城市 s_i 出发，

最多行驶 x_i 公里。

【输出格式】

输出共 $m+1$ 行。

第一行包含一个整数 s_0 ，表示对于给定的 x_0 ，从编号为 s_0 的城市出发，小 A 开车行驶的路程总数与小 B 行驶的路程总数的比值最小。

接下来的 m 行，每行包含 2 个整数，之间用一个空格隔开，依次表示在给定的 s_i 和 x_i 下小 A 行驶的里程总数和小 B 行驶的里程总数。

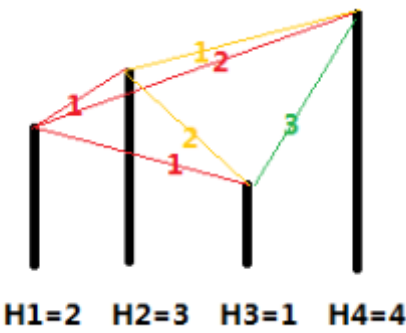
【输入样例 1】

```
4
2 3 1 4
3
4
1 3
2 3
3 3
4 3
```

【输出样例 1】

```
1
1 1
2 0
0 0
0 0
```

【样例解释 1】



各个城市的海拔高度以及两个城市间的距离如上图所示。

如果从城市 1 出发，可以到达的城市为 2,3,4，这几个城市与城

市 1 的距离分别为 1,1,2，但是由于城市 3 的海拔高度低于城市 2，所以我们认为城市 3 离城市 1 最近，城市 2 离城市 1 第二近，所以小 A 会走到城市 2。到达城市 2 后，前面可以到达的城市为 3,4，这两个城市与城市 2 的距离分别为 2,1，所以城市 4 离城市 2 最近，因此小 B 会走到城市 4。到达城市 4 后，前面已没有可到达的城市，所以旅行结束。

如果从城市 2 出发，可以到达的城市为 3,4，这两个城市与城市 2 的距离分别为 2,1，由于城市 3 离城市 2 第二近，所以小 A 会走到城市 3。到达城市 3 后，前面尚未旅行的城市为 4，所以城市 4 离城市 3 最近，但是如果要到达城市 4，则总路程为 $2+3=5>3$ ，所以小 B 会直接在城市 3 结束旅行。

如果从城市 3 出发，可以到达的城市为 4，由于没有离城市 3 第二近的城市，因此旅行还未开始就结束了。

如果从城市 4 出发，没有可以到达的城市，因此旅行还未开始就结束了。

【输入样例 2】

```
10
4 5 6 1 2 3 7 8 9 10
7
10
1 7
2 7
3 7
4 7
5 7
6 7
7 7
8 7
9 7
10 7
```

【输出样例 2】

```
2
3 2
```

2 4
2 1
2 4
5 1
5 1
2 1
2 0
0 0
0 0

【样例解释 2】

当 $x=7$ 时，如果从城市 1 出发，则路线为 $1 \rightarrow 2 \rightarrow 3 \rightarrow 8 \rightarrow 9$ ，小 A 走的距离为 $1+2=3$ ，小 B 走的距离为 $1+1=2$ 。（在城市 1 时，距离小 A 最近的城市是 2 和 6，但是城市 2 的海拔更高，视为与城市 1 第二近的城市，所以小 A 最终选择城市 2；走到 9 后，小 A 只有城市 10 可以走，没有第二选择可以选，所以没法做出选择，结束旅行）

如果从城市 2 出发，则路线为 $2 \rightarrow 6 \rightarrow 7$ ，小 A 和小 B 走的距离分别为 2,4。

如果从城市 3 出发，则路线为 $3 \rightarrow 8 \rightarrow 9$ ，小 A 和小 B 走的距离分别为 2,1。

如果从城市 4 出发，则路线为 $4 \rightarrow 6 \rightarrow 7$ ，小 A 和小 B 走的距离分别为 2,4。

如果从城市 5 出发，则路线为 $5 \rightarrow 7 \rightarrow 8$ ，小 A 和小 B 走的距离分别为 5,1。

如果从城市 6 出发，则路线为 $6 \rightarrow 8 \rightarrow 9$ ，小 A 和小 B 走的距离分别为 5,1。

如果从城市 7 出发，则路线为 $7 \rightarrow 9 \rightarrow 10$ ，小 A 和小 B 走的距离分别为 2,1。

如果从城市 8 出发，则路线为 $8 \rightarrow 10$ ，小 A 和小 B 走的距离分别为 2,0。

如果从城市 9 出发，则路线为 9，小 A 和小 B 走的距离分别为 0,0（旅行一开始就结束了）。

如果从城市 10 出发，则路线为 10，小 A 和小 B 走的距离分别为

0,0。

从城市 2 或者城市 4 出发小 A 行驶的路程总数与小 B 行驶的路程总数的比值都最小，但是城市 2 的海拔更高，所以输出第一行为 2。

【数据范围】

对于 30% 的数据，有 $1 \leq n \leq 20, 1 \leq m \leq 20$ ；

对于 40% 的数据，有 $1 \leq n \leq 100, 1 \leq m \leq 100$ ；

对于 50% 的数据，有 $1 \leq n \leq 100, 1 \leq m \leq 1000$ ；

对于 70% 的数据，有 $1 \leq n \leq 1000, 1 \leq m \leq 104$ ；

对于 100% 的数据： $1 \leq n, m \leq 10^5, -10^9 \leq h_i \leq 10^9, 1 \leq s_i \leq n, 0 \leq x_i \leq 10^9$

数据保证 h_i 互不相同。

文本编辑器(editor.cpp)

【问题描述】

九发明了一个完美的文本编辑器。这个编辑器拥有两个光标(cursor)，所以九能够同时 在两处地方插入和删除文本。这个编辑器除了正常的编辑功能以外，还有一些只有九才知道用处的功能，例如翻转两个光标之间的文本。某一天，九把自己的完美文本编辑器给弄丢了，但是她还有好多好多文本需要处理。于是她想请聪明又智慧的你帮她实现完美文本编辑器的一些功能。

功能列表如下：

功能名称	命令格式	说明
< (move left)	< w	w 为一个字符，“L”或“R”，表示左光标还是右光标（下同）。 该命令将选定光标向左移动，如果已经是最左端则不移动。 命令执行成功时输出“T”，若光标已经在最左端，则输出“F”。
> (move right)	> w	w 同上。 与< 命令不同的是，该命令将光标向右移动。 命令执行成功时输出“T”，若光标已经在最右端，则输出“F”。
I (insert)	I w c	w 同上。 c 是一个可见字符($33 \leq \text{ascii 码} \leq 126$)，代表在该光标左侧插入该字符。 该命令始终输出“T”。
D (delete)	D w	w 同上。 代表删除该光标右侧的一个字符。 命令执行成功时输出“T”，若光标右侧没有字符输出“F”。
R (reverse)	R	代表翻转左光标和右光标之间的字符。 该命令只有左光标在右光标左侧时才能执行。 (两光标重合时也不能执行) 命令执行成功时输出“T”，否则输“F”。
S (show)	S	代表显示当前处理的文本。 该命令只输出文本，不输出“T”和“F”。

开始时文本编辑器中有一定内容，左光标在第一个字符左，右光标在最后一个字符右。

注意：在插入和删除操作中，没有被操作的光标与文本的相对左右位置保持不变。**特别地，若两个光标重叠，操作后也仍然重叠**

【输入格式】

第一行是初始时文本编辑器内容。

第二行是一个正整数 N，N 表示操作次数。

接下来有 N 行，每行有一个命令，命令格式如上方表格

【输出格式】

对于每个命令，按上方表格要求执行并输出。

【输入样例】

goodykc

11

I R u

I R I

> L

> L

> L

> L

RD

R

< R

D R

S

【输出样例】

T

T

T

T

T

T

T

F

T

T

goodluck

【样例解释】

[goodykc]

[goodykcu]

[goodykcul]

g[oodykcul]

go[odykcul]

goo[dykcul]

good[ykcul]

good[lucky]

good[lucky]（光标右边没有字符， 失败删除）

good[luck]y

good[luck]

goodluck

【数据范围】

对于 40%的数据： $1 \leq N$ ，初始文本长度 ≤ 100 ，数据包含翻转(Reverse)操作；

对于 20%的数据： $1 \leq N$ ，初始文本长度 ≤ 100000 ，数据不包含翻转(Reverse)操作；

另有 20%的数据： $1 \leq N$ ，初始文本长度 ≤ 100000 ，数据包含翻(Reverse)操作；

对于 100%的数据： $1 \leq N$ ，初始文本长度 $\leq 4 \times 10^6$ ，输出文件大小 $\leq 20\text{MB}$ ；